Method 1: Utilizing dummy variables

Background:

Previously, we utilized a plackett-luce model in order to predict finishing position. All the **predictors needed to be standardized**. This was due to the fact that their **scales were different and comparing them directly would lead to bias**. This caused the main problem as this can be done, but one of our predictors: **qualifying position, is categorical and ordinal, meaning that it isn't continuous**. So s**tandardizing this, takes away the main meaning of qualifying which is the order.** To combat this, a **dummy variable** needed to be created meaning that instead of the qualifying position being one predictor, it would be 19 due to the comparison to the dummy.

This again created a new problem. Along with these 19 predictors, we **created a quali x overtake interaction as a predictor, which meant there would be another 19 predictors for this**. Along with the 5 continuous ones we had, this meant there would be **43 predictors** total, but there are **only 23 races, and 20 drivers per race**. This would **make qualifying position nearly fully deterministic for finishing position. It creates quasi-complete separation**.

When the model is given one coefficient and one interaction per grid slot, along with no regularization, the optimizer learns that it can predict this race perfectly by sending the coefficients to infinity as we will see, causing qualifying position to almost perfectly order finishing position in many races.

Results:

Figures 1 and 2: Preprocessing

```
Missing values by predictor:
Quali_1.0                 0
Quali_2.0                 0
Quali_3.0                 0
Quali_4.0                 0
Quali_5.0                 0
Quali_6.0                 0
Quali_7.0                 0
Quali_8.0                 0
Quali_9.0                 0
Quali_10.0                0
Quali_11.0                0
Quali_12.0                0
Quali_13.0                0
Quali_14.0                0
Quali_15.0                0
Quali_16.0                0
Quali_17.0                0
Quali_18.0                0
Quali_19.0                0
Quali_20.0                0
```

```
Quali_1.0_x_Overtake     0
Quali_2.0_x_Overtake     0
Quali_3.0_x_Overtake     0
Quali_4.0_x_Overtake     0
Quali_5.0_x_Overtake     0
Quali_6.0_x_Overtake     0
Quali_7.0_x_Overtake     0
Quali_8.0_x_Overtake     0
Quali_9.0_x_Overtake     0
Quali_10.0_x_Overtake    0
Quali_11.0_x_Overtake    0
Quali_12.0_x_Overtake    0
Quali_13.0_x_Overtake    0
Quali_14.0_x_Overtake    0
Quali_15.0_x_Overtake    0
Quali_16.0_x_Overtake    0
Quali_17.0_x_Overtake    0
Quali_18.0_x_Overtake    0
Quali_19.0_x_Overtake    0
Quali_20.0_x_Overtake    0
DriverForm               0
TeamStrength             0
DriverDNF_Rate           0
TeamDNF_Rate             0
OvertakeIndex            0
```

After preprocessing, we see that there are no zero values for any of our predictors. Along with this our **qualifying positions are floats, meaning pandas sees 20 distinct float categories**

which is why there are 45 and not 43 predictors in the list. Either way this is too much with only 23 races.

Figure 3: Coefficient interpretation

	Predictor	Coefficient	StdError	z_value	p_value
0	Quali_1.0	1.920532e+11	NaN	NaN	NaN
1	Quali_2.0	2.365837e+11	NaN	NaN	NaN
2	Quali_3.0	1.049020e+11	NaN	NaN	NaN
3	Quali_4.0	2.338309e+11	NaN	NaN	NaN
4	Quali_5.0	1.736500e+11	NaN	NaN	NaN
5	Quali_6.0	1.144875e+11	NaN	NaN	NaN
6	Quali_7.0	2.007203e+11	NaN	NaN	NaN
7	Quali_8.0	6.985635e+10	NaN	NaN	NaN
8	Quali_9.0	7.970303e+10	NaN	NaN	NaN
9	Quali_10.0	-5.764072e+10	NaN	NaN	NaN
10	Quali_11.0	-2.182384e+10	NaN	NaN	NaN
11	Quali_12.0	-3.346830e+10	NaN	NaN	NaN
12	Quali_13.0	-3.662034e+10	NaN	NaN	NaN
13	Quali_14.0	-1.232665e+11	NaN	NaN	NaN
14	Quali_15.0	-4.481386e+10	NaN	NaN	NaN
15	Quali_16.0	-4.026156e+10	NaN	NaN	NaN
16	Quali_17.0	-2.228364e+09	NaN	NaN	NaN
17	Quali_18.0	-2.485955e+10	NaN	NaN	NaN
18	Quali_19.0	-7.715143e+10	NaN	NaN	NaN
19	Quali_20.0	1.651983e+10	NaN	NaN	NaN

All predictors—including continuous variables—exhibit **NaN standard errors and test statistics.** This occurs because the **Hessian matrix is singular or indefinite**, indicating that the **coefficients are not uniquely identifiable**. This is a **direct consequence of quasi-complete separation** (one or more independent variables can predict outcome variables with almost perfect accuracy, but not perfectly).

Figures 4 and 5: Model Evaluation metrics

```
Log-likelihood: 277115036764265.8
Mean Spearman: 0.45590062111801244
Mean Kendall: 0.31807780320366136
Top-3 accuracy: 0.3623188405797101
```

1	Year	Round	Driver	alRacePos	PredictedRank	PL_Win_Probability	Race_LogLikelihood	Approx_PL_Top3_Probability
2	2024	1	VER	1	7		-inf	
3	2024	1	PER	2	6		-inf	
4	2024	1	SAI	3	3		-inf	
5	2024	1	LEC	4	2		-inf	
6	2024	1	RUS	5	5		-inf	
7	2024	1	NOR	6	1		-inf	
8	2024	1	HAM	7	16		-inf	
9	2024	1	PIA	8	9		-inf	
10	2024	1	ALO	9	4		-inf	
11	2024	1	STR	10	10		-inf	
12	2024	1	ZHO	11	13		-inf	
13	2024	1	MAG	12	17		-inf	
14	2024	1	RIC	13	20	0	-inf	0

The exponential utility function in the PL likelihood **overflows due to diverging coefficients**, causing **probabilities to collapse to zero and race-level log-likelihoods to reach negative infinity**. Rank-based evaluation metrics also collapse, indicating that the **model is no longer learning meaningful ranking structure**.

Conclusion:

The primary issue in this specification is the **number and structure of predictors**, not the Plackett–Luce model itself. Assigning a unique coefficient to each qualifying position allows the model to nearly deterministically order finishing positions within races, leading to quasi-complete separation. This is a **design-matrix identifiability problem**, n**ot a failure of the ranking model**. Any sufficiently expressive ranking model would exhibit similar behavior under this specification.

<u>Method 2: Dropping interactions (quali x overtake)</u>

Background

Dropping the interaction terms could be another potential method as it would greatly reduce the number of predictors we have. In method 1, the linear predictor looked like:

$$\eta_i = \underbrace{\sum_{k=1}^{19} \beta_k \, 1(\text{Quali} = k)}_{\text{grid effects}} + \underbrace{\sum_{k=1}^{19} \gamma_k \, 1(\text{Quali} = k) \times \text{Overtake}}_{\text{interaction}} + \underbrace{\delta^\top X_{\text{continuous}}}_{\text{5 continuous predictors}}$$

Now, with dropping the interactions terms:

$$\eta_i = \sum_{k=1}^{19} \beta_k \, 1(\text{Quali} = k) + \delta^\top X_{\text{continuous}}$$

Our parameters drop from **43 to 24, increasing the likelihood that the Hessian matrix is invertible and reducing—but not eliminating—the risk of quasi-complete separation.**

Results:

Figure 1: Preprocessing

```
Preprocessing complete.
Missing values by predictor:
Quali_1.0        0
Quali_2.0        0
Quali_3.0        0
Quali_4.0        0
Quali_5.0        0
Quali_6.0        0
Quali_7.0        0
Quali_8.0        0
Quali_9.0        0
Quali_10.0       0
Quali_11.0       0
Quali_12.0       0
Quali_13.0       0
Quali_14.0       0
Quali_15.0       0
Quali_16.0       0
Quali_17.0       0
Quali_18.0       0
Quali_19.0       0
Quali_20.0       0
DriverForm       0
TeamStrength     0
DriverDNF_Rate   0
TeamDNF_Rate     0
OvertakeIndex    0
```

We see after preprocessing, there are no missing values, and we now have 24 predictors

Figure 2: Coefficient interpretation

```
Constructed 23 race rankings.
         Predictor   Coefficient    StdError       z_value  p_value
0        Quali_1.0  9.892145e+06  223.405352  44278.908241      0.0
1        Quali_2.0  9.334122e+06  210.803485  44278.782605      0.0
2        Quali_3.0  7.822008e+06  176.654117  44278.663300      0.0
3        Quali_4.0  9.241318e+06  208.708344  44278.622645      0.0
4        Quali_5.0  8.536470e+06  192.789584  44278.686605      0.0
5        Quali_6.0  6.316533e+06  142.655565  44278.212237      0.0
6        Quali_7.0  6.111499e+06  138.024850  44278.251784      0.0
7        Quali_8.0  6.271751e+06  141.644111  44278.230313      0.0
8        Quali_9.0  4.566203e+06  103.127727  44277.163337      0.0
9       Quali_10.0  9.367299e+06  211.552196  44278.901431      0.0
10      Quali_11.0  3.264492e+06   73.730930  44275.748804      0.0
11      Quali_12.0  2.973592e+06   67.162592  44274.532261      0.0
12      Quali_13.0  1.330367e+06   30.059941  44257.125682      0.0
13      Quali_14.0  1.716146e+06   38.768842  44266.127337      0.0
14      Quali_15.0  1.889612e+06   42.685605  44268.123372      0.0
15      Quali_16.0  8.946728e+04    2.231334  40095.872708      0.0
16      Quali_17.0 -4.534457e+05   10.281422 -44103.401452      0.0
17      Quali_18.0  1.032653e+06   23.341111  44241.823294      0.0
18      Quali_19.0 -5.820029e+05   13.176593 -44169.453268      0.0
19      Quali_20.0  3.534501e+06   79.825616  44277.785014      0.0
20      DriverForm -3.091622e+06   69.824717 -44276.899919      0.0
21    TeamStrength -2.184387e+07  493.320024 -44279.308407      0.0
22  DriverDNF_Rate  1.812651e+06   40.937614  44278.376814      0.0
23    TeamDNF_Rate -3.033535e+06   68.513680 -44276.341692      0.0
24   OvertakeIndex  1.847775e+01    1.000000     18.477751      0.0
```

Although we don't have NA values anymore, our coefficients are still "exploding". Our z values are around 40,000 and p values are zero. This means the model is still trying to perfectly order the finishing position using qualifying alone.

Figures 3 and 4: Model Evaluation metrics

```
Log-likelihood: 2118324294.5758407
Mean Spearman: -0.5029094475318729
Mean Kendall: -0.3322654462242563
Top-3 accuracy: 0.014492753623188404
```

A	B	C	D	E	F	G	H	I
Year	Round	Driver	alRacePos	edictedRa	Win_Proba	_LogLikeli	PL_Top3_Probability	
2024	1	VER	1	19	0	-inf	0	
2024	1	PER	2	20	0	-inf	0	
2024	1	SAI	3	18	0	-inf	0	
2024	1	LEC	4	14		-inf		
2024	1	RUS	5	12		-inf		
2024	1	NOR	6	15		-inf		
2024	1	HAM	7	17	0	-inf	0	
2024	1	PIA	8	16		-inf		
2024	1	ALO	9	13		-inf		
2024	1	STR	10	10		-inf		
2024	1	ZHO	11	7		-inf		
2024	1	MAG	12	5		-inf		
2024	1	RIC	13	4		-inf		

Race-level log-likelihoods remain at negative infinity, and r**ank correlation metrics are worse than random, with negative Spearman and Kendall values**. The **model predicts nearly the same ranking across races and exhibits extreme overconfidence.**

Conclusion:

Removing interaction terms reduced model dimensionality but did not fully resolve quasi-complete separation. Qualifying position dummies are giving the model too much freedom. Again, this is not a Plackett-Luce failure, it is a design-matrix identifiability problem. Any ranking model would have this similar issue. This indicates that the issue lies in predictor encoding rather than model specification, motivating further structural constraints through binning and regularization.

Method 3: Bin Qualifying positions

Background

In order to address quasi-complete separation, our main problem is that there are too many qualifying predictors due to it being categorical and ordinal and us having to transform them into dummy variables. For this, qualifying positions was put into three ordinal bins.

- Front: Positions 1-3
- Mid: Positions 3-10
- Back: Positions 11-20

This **reduces the number of qualifying-related parameters from 19 to 2**, while preserving the ordinal nature of starting position. Importantly, these **bins are not standardized**, as they **remain categorical rather than continuous**

Results:

Figure 1: Preprocessing

```
Preprocessing complete.
Missing values by predictor:
QualiBin_Front    0
QualiBin_Mid      0
DriverForm        0
TeamStrength      0
DriverDNF_Rate    0
TeamDNF_Rate      0
OvertakeIndex     0
```

Now, after pre-processing we only have 7 parameters instead of 24, again with no missing values.

Figure 2: Coefficient interpretation

	Predictor	Coefficient	StdError	z_value	p_value
0	QualiBin_Front	2.752071e+13	NaN	NaN	NaN
1	QualiBin_Mid	8.109504e+13	NaN	NaN	NaN
2	DriverForm	-2.657841e+12	NaN	NaN	NaN
3	TeamStrength	-2.464382e+13	NaN	NaN	NaN
4	DriverDNF_Rate	-1.005213e+13	NaN	NaN	NaN
5	TeamDNF_Rate	8.227856e+11	NaN	NaN	NaN
6	OvertakeIndex	7.342929e+08	NaN	NaN	NaN

Unfortunately, when we look at our coefficients, they are still exploding, and unlike before, we now also have another problem with the standard error, z values, and p values all being NaN. This shows us that we still didn't fully address quasi-complete separation so an additional step needs to be taken.

Figures 3 and 4: Model Evaluation metrics

```
Log-likelihood: 1.4982751944268606e+16
Mean Spearman: 0.11180124223602485
Mean Kendall: 0.06636155606407323
Top-3 accuracy: 0.014492753623188404
```

1	Year	Round	Driver	alRacePos	edictedRa	Win_Proba	_LogLikeli	PL_Top3_Probability
2	2024	1	VER	1	20	0	-inf	0
3	2024	1	PER	2	7		-inf	
4	2024	1	SAI	3	6		-inf	
5	2024	1	LEC	4	15		-inf	
6	2024	1	RUS	5	8		-inf	
7	2024	1	NOR	6	3		-inf	
8	2024	1	HAM	7	5		-inf	
9	2024	1	PIA	8	4		-inf	
10	2024	1	ALO	9	2		-inf	
11	2024	1	STR	10	18		-inf	
12	2024	1	ZHO	11	12		-inf	
13	2024	1	MAG	12	14		-inf	
14	2024	1	RIC	13	10		-inf	
15	2024	1	TSU	14	11		-inf	
16	2024	1	ALB	15	17		-inf	
17	2024	1	HUL	16	1		-inf	
18	2024	1	OCO	17	9		-inf	
19	2024	1	GAS	18	19		-inf	
20	2024	1	BOT	19	13		-inf	
21	2024	1	SAR	20	16		-inf	

Again, we can see here that our model evaluation is quite low. Our mean spearman and

kendall aren't negative like they were in method 2, but 0.11 and 0.07 respectively are quite low. Our log-likihood exploded too with it being to the power of 16 and our top 3 accuracy is 0.01. When looking at the datasheet, we see we get -inf for the log-likelihood and our win_prob and PL_top3_probability doesn't even show up except for the first place driver.

Conclusion:

From what we can see, while reducing the parameters on paper should help, we still see the problem of quasi-complete separation occurring. This tells us that there is still something that needs to be added to the model to address this problem.

Regularization (L2 or L1), could immensely help our model.

- L2 Regularization (Ridge): Shrink coefficients to 0
- L1 Regularization (Lasso): Sets coefficients to 0

Adding these should be our next step.

Method 4: Bin qualifying positions + Regularization (L2 Penalty - Ridge)

Background

To directly address quasi-complete separation, the qualifying position was discretized into three ordinal bins as mentioned in method 2..

Next, **even with bins Plackett-luce can still inflate coefficients, overreact to strong predictors, or become numerically unstable**. For these cases, we add **L2 regularization (ridge)**:

$$\text{Loss} = -\ell(\beta) + \lambda \sum_j \beta_j^2$$

Doing this, penalizes large coefficients, prevents β from going to infinity, and makes the Hessian invertible.

Results:

Figure 1: Preprocessing

```
Preprocessing complete.
Missing values by predictor:
QualiBin_Front     0
QualiBin_Mid       0
DriverForm         0
TeamStrength       0
DriverDNF_Rate     0
TeamDNF_Rate       0
OvertakeIndex      0
```

After pre-processing, there are no missing values, and our parameters have been reduced to 7.

Figure 2: Coefficient interpretation

```
Constructed 23 race rankings.
        Predictor  Coefficient  StdError     z_value   p_value
0  QualiBin_Front    34.999979  0.422452   82.849670  0.000000
1    QualiBin_Mid    80.499974  0.632902  127.191920  0.000000
2      DriverForm    -3.256428  0.006875 -473.645638  0.000000
3    TeamStrength     3.096051  0.017170  180.317292  0.000000
4  DriverDNF_Rate    -0.854199  0.008950  -95.440694  0.000000
5    TeamDNF_Rate    -2.230263  0.021165 -105.376171  0.000000
6   OvertakeIndex     0.000182  0.017071    0.010677  0.991481
```

The **qualifying bin coefficients are large and positive, indicating that starting near the front of the grid strongly increases relative utility**. This behavior is expected: **PL coefficients represent relative utilities, not probabilities**, and **utility differences are exponentiated when computing ranking probabilities.**

Driver form exhibits a negative coefficient, indicating that poorer recent performance is associated with worse race outcomes, while team strength is positively associated with finishing position. Driver- and team-level DNF rates are negative, reflecting increased risk of poor finishes.

Notably, **once conditioning on qualifying bins, overtaking difficulty contributes little additional ranking power, resulting in a near-zero coefficient with a high p-value**.

Figures 3 and 4: Model Evaluation metrics

```
Log-likelihood: 7623.805245678994
Mean Spearman: 0.6037267080745341
Mean Kendall: 0.4279176201372999
Top-3 accuracy: 0.21739130434782608
```

	A	B	C	D	E	F	G	H
1	Year	Round	Driver	alRacePos	PredictedRank	PL_Win_Probability	Race_LogLikelihood	Approx_PL_Top3_Probability
2	2024	1	VER	1	8	4.164E-19	-610.4409411	1.2492E-18
3	2024	1	PER	2	1	0.994760586	-610.4409411	1
4	2024	1	SAI	3	6	0.000105765	-610.4409411	0.000317295
5	2024	1	LEC	4	9	6.42801E-22	-610.4409411	1.9284E-21
6	2024	1	RUS	5	10	5.00872E-23	-610.4409411	1.50262E-22
7	2024	1	NOR	6	4	0.000190634	-610.4409411	0.000571902
8	2024	1	HAM	7	5	0.000128281	-610.4409411	0.000384844
9	2024	1	PIA	8	2	0.004249372	-610.4409411	0.012748116
10	2024	1	ALO	9	3	0.000565308	-610.4409411	0.001695924
11	2024	1	STR	10	11	3.26362E-40	-610.4409411	9.79086E-40
12	2024	1	ZHO	11	16	3.9733E-42	-610.4409411	1.19199E-41
13	2024	1	MAG	12	19	1.04001E-42	-610.4409411	3.12002E-42
14	2024	1	RIC	13	15	7.76622E-42	-610.4409411	2.32987E-41
15	2024	1	TSU	14	17	3.9733E-42	-610.4409411	1.19199E-41
16	2024	1	ALB	15	14	8.27128E-42	-610.4409411	2.48138E-41
17	2024	1	HUL	16	7	5.36696E-08	-610.4409411	1.61009E-07
18	2024	1	OCO	17	12	2.21564E-40	-610.4409411	6.64691E-40
19	2024	1	GAS	18	13	4.40524E-41	-610.4409411	1.32157E-40
20	2024	1	BOT	19	18	2.0328E-42	-610.4409411	6.09839E-42
21	2024	1	SAR	20	20	5.32082E-43	-610.4409411	1.59625E-42

Our rank correlations of Spearman (0.6), and Kendall (0.43) improve, indicating the model captures meaningful ordinal structure.. Our log-likelihood has reduced but is still quite large, due to the cumulative nature of ranking likelihoods. We also notice that for PL_Win_Probabilitiy, one driver has about 0.99 while everyone else is 1e-20, which is common PL behavior. PL win probabilities are very sharp.

Conclusion:

To address quasi-complete separation caused by assigning a unique parameter to each grid position, qualifying position was discretized into three ordinal bins (Front, Mid, Back). Additionally, an L2 penalty was applied to stabilize coefficient estimation.

While this helped, coefficient magnitudes and win probabilities still remain sharp, which is inherently true to Plackett Luce models, where utilities are exponentiated from ranking probabilities. These effects arise from shared structure across drivers rather than per-position memorization like before.

Exploring alternative binning strategies for qualifying position,and incorporating cross-validation across races could be used to improve this. Along with this, extending the model to use hierarchical or Bayesian plackett_luce formulations could allow partial pooling across drivers and teams, further smoothing utility estimates while retaining interpretability.

Method 5: Bin qualifying positions + Regularization (L1 Penalty - Lasso)

Background:

Continuing with the bins, to investigate whether further sparsity could improve interpretability, I replaced the L2 penalty with L1 (Lasso) penalty. Ridge (L2), what we used in method 3, shrinks the coefficient close to zero, while L1 (Lasso), shrinks weak predictors to be exactly zero. This could help identify which race-level or driver-level factors contribute marginally beyond qualifying position.

Results:

Figures 2: Preprocessing

```
Preprocessing complete.
Missing values by predictor:
QualiBin_Front    0
QualiBin_Mid      0
DriverForm        0
TeamStrength      0
DriverDNF_Rate    0
TeamDNF_Rate      0
OvertakeIndex     0
```

After pre-processing similarly to L2, we still have 7 parameters, with no missing values.

Figure 2: Coefficient interpretation

```
Constructed 23 race rankings.
        Predictor   Coefficient     StdError       z_value  p_value
0  QualiBin_Front  3.791944e+07   502.617701  75443.898462      0.0
1    QualiBin_Mid  1.118872e+08  1483.045567  75444.202217      0.0
2      DriverForm -3.693255e+06    48.960251 -75433.736223      0.0
3    TeamStrength -3.396840e+07   450.245765 -75444.121898      0.0
4  DriverDNF_Rate -1.386748e+07   183.812560 -75443.598316      0.0
5    TeamDNF_Rate  1.125100e+06    14.937240  75321.804657      0.0
6   OvertakeIndex -5.479193e+04     1.238489 -44240.948650      0.0
```

What we see here, is that **Lasso did indeed eliminate some predictors by setting them very close to zero, but the coefficients for the variables that remained again "exploded", with high exponent powers**. Along with that our standard errors shot up again. This behavior occurs due to the **Plackett-luce likelihood increasing exponentially with utility differences, while the L1 penalty only grows linearly.** Due to this, only a small subset of predictors is retained, and the optimizer can increase their coefficients without sufficient penalty, which again reintroduced quasi-complete separation.

Figures 3 and 4: Model Evaluation metrics

```
Log-likelihood: 20647946678.22704
Mean Spearman: 0.11173586139261196
Mean Kendall: 0.06590389016018307
Top-3 accuracy: 0.014492753623188404
```

1	Year	Round	Driver	ActualRacePosition	PredictedRank	L_Win_Probabili	Race_LogLikelihood	pprox_PL_Top3_Probabilit
2	2024	1	VER	1	20	0	-inf	0
3	2024	1	PER	2	7		-inf	
4	2024	1	SAI	3	6		-inf	
5	2024	1	LEC	4	15		-inf	
6	2024	1	RUS	5	8		-inf	
7	2024	1	NOR	6	3		-inf	
8	2024	1	HAM	7	5		-inf	
9	2024	1	PIA	8	4		-inf	
10	2024	1	ALO	9	2		-inf	
11	2024	1	STR	10	18		-inf	
12	2024	1	ZHO	11	12		-inf	
13	2024	1	MAG	12	14		-inf	
14	2024	1	RIC	13	10		-inf	
15	2024	1	TSU	14	11		-inf	
16	2024	1	ALB	15	17		-inf	
17	2024	1	HUL	16	1		-inf	
18	2024	1	OCO	17	9		-inf	
19	2024	1	GAS	18	19		-inf	
20	2024	1	BOT	19	13		-inf	
21	2024	1	SAR	20	16		-inf	

The model evaluation metrics reflect this instability. Our log-likelihood becomes very large due to inflated utilities, and our rank based metrics are much smaller than with L2. Additionally, we see unstable log-likelihood estimates with -inf. This shows that the Lasso model failed to produce stable results.

Conclusion:

Although LASSO performs effective variable selection, it is **insufficient on its own to control coefficient magnitude** in Plackett–Luce ranking models with strong predictors such as

qualifying position. The **linear growth of the L1 penalty cannot counteract the exponential nature of the PL likelihood, allowing retained coefficients to diverge and reintroducing numerical instability and quasi-complete separation.**